

Assignment 5: Advanced Multi-Client Chat Server using TCP

Jitupan Mondal
Roll Number: EEB24023

July 4, 2026

1 Objective

This project extends the multi-client TCP chat server built in Assignment 4 by adding advanced client management, private messaging, an online user list, persistent chat history, and detailed performance evaluation. The goal is to demonstrate concurrent server design, server-side message routing, and state management in a socket-based client-server architecture.

2 System Architecture

The system follows a centralized client-server model. A single TCP server (`server.py`) listens on port 5000 and spawns one dedicated thread per connected client to handle communication concurrently. Each client (`client.py`) maintains two threads: one for sending user input and one for receiving broadcast or private messages in real time. All routing decisions (broadcast vs. private) are made exclusively by the server.

System Details

- **OS:** Linux jitupan-mondal-QEMU-Virtual-Machine 7.0.0-22-generic (Ubuntu, aarch64)
- **Python Version:** 3.14.4
- **Network Interface:** enp0s1, IP 192.168.64.2/24

3 Design Improvements over Assignment 4

- **Client Management:** Extended the client registry from a simple socket-to-username mapping to a structured record storing username, IP, port, login time, and online status.
- **Private Messaging:** Introduced server-side routing using a username-to-socket lookup, enabling the `/msg <username> <message>` command to deliver messages to a single recipient instead of broadcasting.
- **Online User List:** Added the `/list` command, which replies only to the requesting client with the current set of connected usernames.
- **Persistent Chat History:** Replaced the flat text log with a structured `chat_history.csv` file containing timestamp, sender, receiver, message type, and message content. On reconnect, the server retrieves and displays the user's last five sent messages.

- **Server Statistics:** Added counters for total messages, broadcast messages, and private messages, printed on server shutdown.

4 Program Design

The server uses a global thread-safe dictionary (`clients`) protected by a `threading.Lock` to store per-client metadata. Incoming messages are parsed for command prefixes (`/msg`, `/list`) before falling back to default broadcast behavior. All chat activity is logged to `chat_history.csv` using Python’s built-in `csv` module for structured, machine-readable storage.

Listing 1: Private messaging routing logic

```
if message.startswith("/msg_"):
    parts = message.split("_", 2)
    target_user, priv_msg = parts[1], parts[2]
    target_sock = find_socket_by_username(target_user)
    if target_sock:
        target_sock.sendall(f"[PM_ from_{username}]_{priv_msg}\n".encode())
        log_chat(username, target_user, "private", priv_msg)
```

5 Experimental Setup

Performance was measured using an automated test client (`script.py`) that connects to the server and sends 50 messages per client. Experiments were repeated with 2, 3, and 4 simultaneous clients inside a Mininet topology (`sudo mn --topo single,5`), with one server host and four client hosts.

6 Results

Table 1: Performance Results (Averaged per Client Count)

Number of Clients	Avg. Delivery Time (ms)	Throughput (msgs/sec)
2	22.51	44.48
3	21.96	45.54
4	22.20	45.10

Table 2: Message Type Distribution

Message Type	Count
Broadcast	1
Private	6

7 Graph Analysis

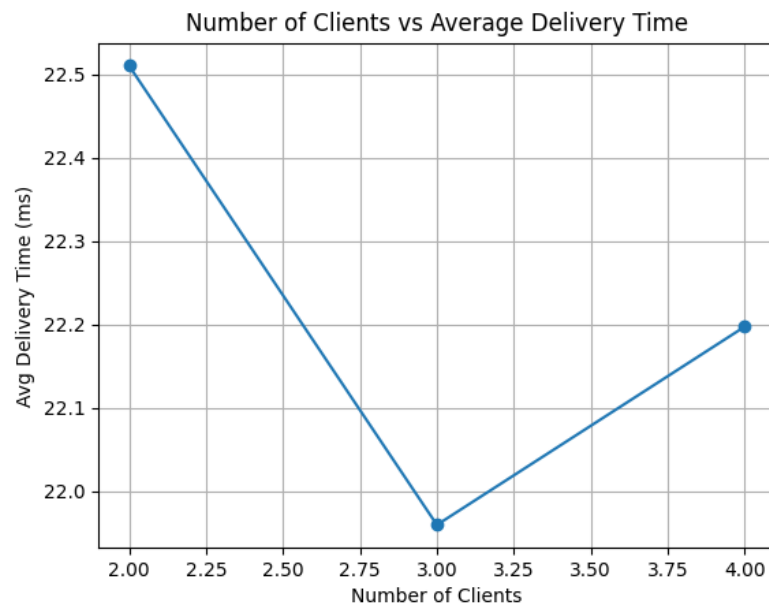


Figure 1: Number of Clients vs Average Delivery Time

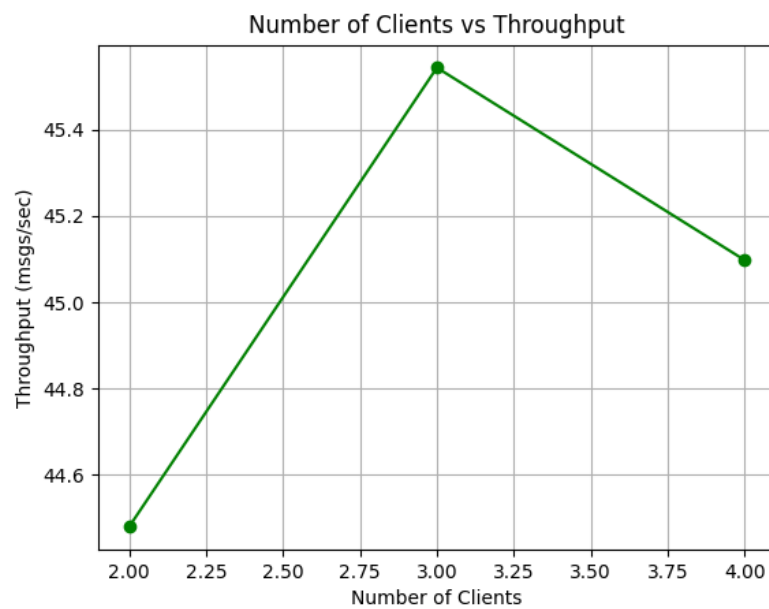


Figure 2: Number of Clients vs Throughput

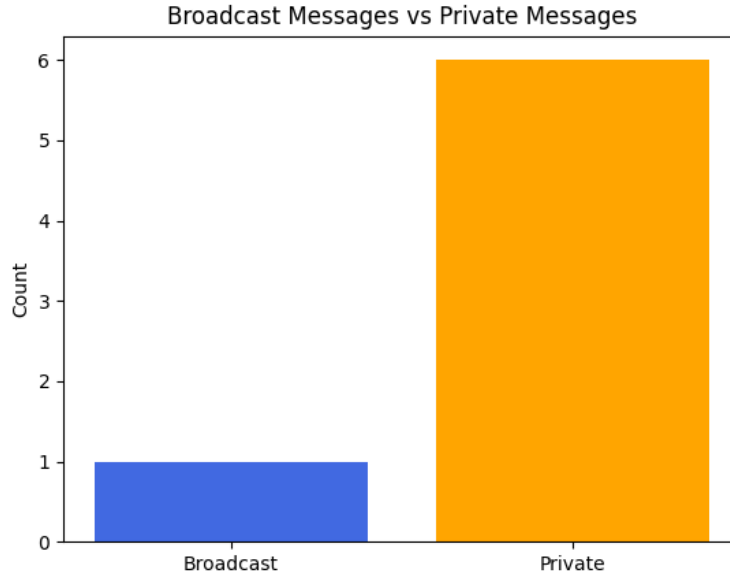


Figure 3: Broadcast Messages vs Private Messages

The average delivery time remained nearly constant (approximately 22 ms) as the number of clients increased from 2 to 4, while throughput stayed stable in the range of 44–46 messages per second. This indicates that the thread-per-client architecture scales well at this small client scale, with no significant degradation in responsiveness.

8 Wireshark Verification

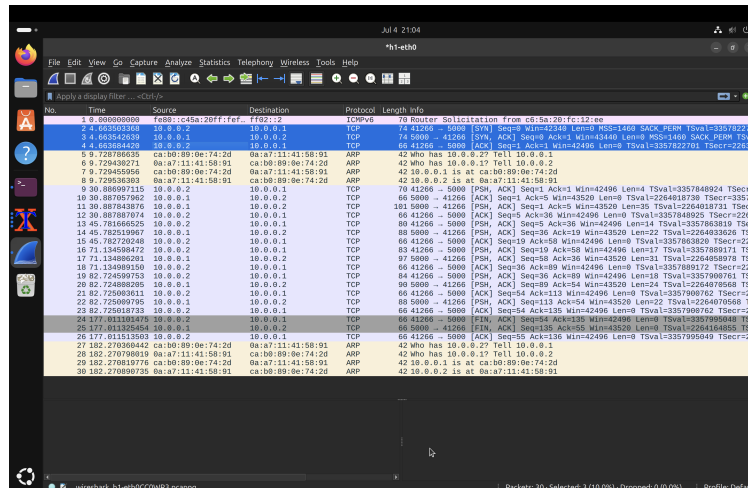


Figure 4: TCP Three-Way Handshake

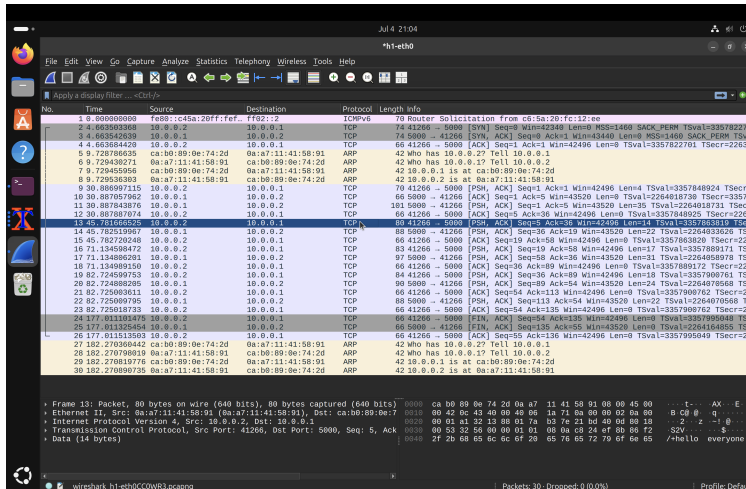


Figure 5: Broadcast Message Packet

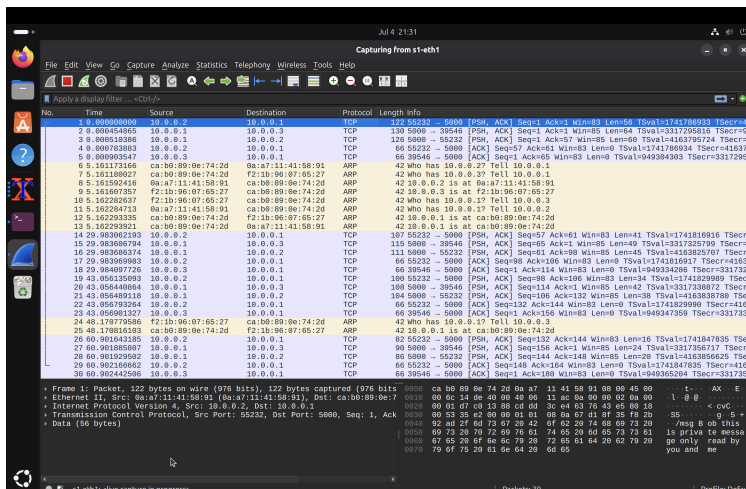


Figure 6: Private Message Packet

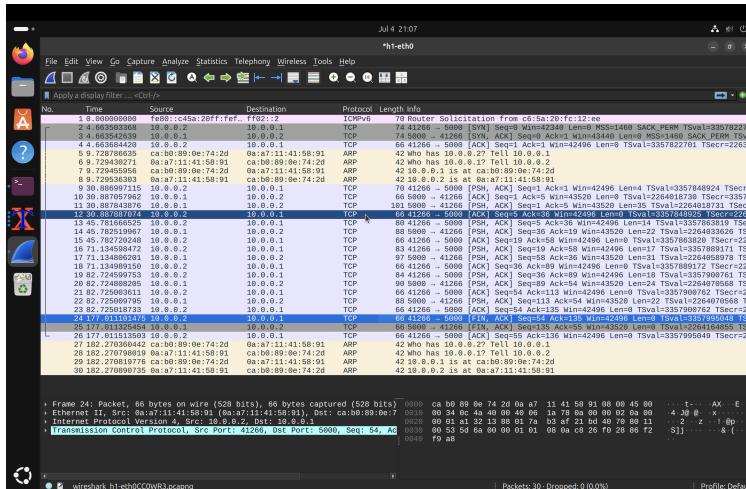


Figure 7: Client Disconnect

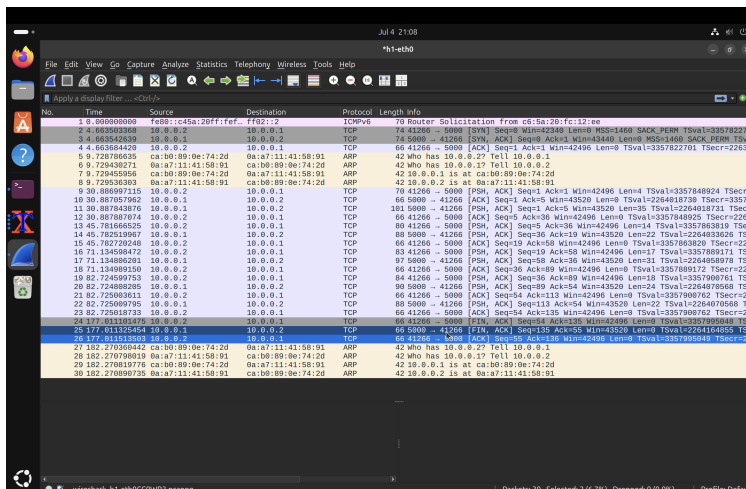


Figure 8: TCP Connection Termination

The captured packets confirm correct TCP behavior: the three-way handshake precedes every client connection, individual PSH, ACK packets correspond to broadcast and private messages sent over port 5000, and the connection cleanly terminates via FIN, ACK exchanges upon client disconnect.

9 Reflection Answers

1. Why is server-side routing necessary for private messaging?

In a TCP-based client-server model, clients only maintain a socket connection to the server, not to each other. The server is the only entity with visibility into all connected clients, so it must inspect the destination username and forward the message to the correct socket. Without server-side routing, private messages would have no path to reach only the intended recipient.

2. How does maintaining user state improve the application?

Storing metadata such as IP, port, login time, and status allows the server to support features like the online user list, private messaging, and statistics reporting. It also enables reconnect handling,

where the server can recognize a returning username and retrieve their prior chat history, improving continuity for the user.

3. How would the application scale to 100 users?

The current thread-per-client model would face increased context-switching overhead and memory usage with 100 simultaneous threads. A more scalable design would use asynchronous I/O (e.g., Python's `asyncio` or `select/epoll`-based event loops) to handle many connections on fewer threads, along with a more efficient username lookup structure than a linear dictionary scan.

4. Which Wireshark packets verified private messaging?

The PSH, ACK packet captured immediately after issuing the `/msg` command, addressed from the server to only the recipient's socket connection, verified that private messages were not broadcast to all clients. Following the TCP stream confirmed the private message content was delivered to a single destination port.

5. What improvements would you implement next?

Future improvements could include message encryption (TLS) for confidentiality, delivery acknowledgments for reliability tracking, a persistent database instead of CSV files for chat history, and graceful handling of duplicate username reconnects.

10 Conclusion

This assignment successfully extended the Assignment 4 chat server with private messaging, enhanced client state management, persistent chat history, and performance evaluation. Experimental results confirm that the server maintains stable delivery times and throughput as client count increases from 2 to 4, and Wireshark captures validate correct TCP-level behavior for both broadcast and private message delivery.